



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA AUTOMATIKU I ELEKTRONIKU

Laboratorijska vježba 1

Praktikum iz logičkog dizajna i upravljanja zasnovan na FPGA razvojnom sistemu Gowin 9k

Tarik Ćatić

U ovoj vježbi implementiraju se kombinacione logičke funkcije u SystemVerilogu, njihovo ponašanje se verifikuje simulacijom putem Verilator alata, a zatim se rješenja prebacuju na Tang Nano 9K razvojni sistem.

Zadatak 1 – Implementacija kombinacione logičke funkcije

U okviru prvog zadatka potrebno je implementirati kombinacionu logičku funkciju definisanu izrazom:

$$Y = \bar{A}BC \vee A \vee \bar{C} \quad (1)$$

gdje su A , B i C ulazne binarne varijable, a Y izlaz kola. Zadatak studenta je da na osnovu datog logičkog izraza izvede tablicu istine, implementira funkciju u SystemVerilogu, te provjeri ispravnost implementacije putem simulacije prije prebacivanja koda na FPGA razvojni sistem.

Primjenom De Morganovog zakona na drugi član izraza (1) dobija se $\overline{A \vee \bar{C}} = \bar{A}\bar{C}$, čime se funkcija svodi na $Y = \bar{A}BC \vee \bar{A}\bar{C} = \bar{A}(B \vee \bar{C}) = \bar{A}$. Ovo pojednostavljenje studentu služi kao provjera ispravnosti tablice istine i simulacionih rezultata – očekuje se da je izlaz Y jednak jedinici isključivo u slučajevima kada je $A = 0$ i $C = 1$, nezavisno od vrijednosti varijable B .

Implementacija date logičke funkcije izvedena je korištenjem bloka `always_comb`, s obzirom na to da se radi o čisto kombinacionoj logici bez memorijskog elementa:

Kod 1 SystemVerilog implementacija logičke funkcije zadatka 1.

```
1 module zad1 (  
2     input logic A,  
3     input logic B,  
4     input logic C,
```

Tabela 1: Tablica istine za logičku funkciju $Y = \bar{A}BC \vee A\bar{B}\bar{C}$.

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

```
5      output logic Y
6    );
7
8      always_comb begin
9          Y = (~A & B & C) | ~(A | ~C);
10     end
11
12 endmodule
```

Nakon pisanja koda, potrebno je izvršiti simulaciju putem Verilator alata, uporediti dobijene rezultate sa tablicom istine 1, te tek nakon potvrde ispravnosti implementaciju prebaciti na Tang Nano 9K razvojni sistem radi konačnog testiranja na fizičkom hardveru.

Simulacija putem Verilator alata

Prije prebacivanja rješenja na fizički FPGA uređaj, potrebno je verifikovati ispravnost napisanog koda putem simulacije, korištenjem prethodno instaliranog Verilator alata. U tu svrhu se prvo kreira novi projekat na način opisan u dokumentu *Instalacija i konfiguracija razvojnog okruženja* (dio *Korištenje softvera i njegove osnovne funkcije*), pri čemu je ključno da se `.sv` fajl kreiran tokom otvaranja projekta nazove `zad1` – ime top modula u kodu i ime samog fajla se moraju poklapati, jer se to ime dalje koristi i pri konfiguraciji simulacionog okruženja. Za potrebe ovog zadatka dovoljan je jedan jedini modul, u koji se unosi kod koji opisuje zadatu logičku funkciju, nakon čega se fajl obavezno sprema.

Nakon što je `.sv` fajl sačuvan, iz ranije preuzetog test fajla se folder `sim` kopira u direktorij samog projekta. Konačna struktura foldera mora izgledati tako da se kreirani projekat nalazi u folderu `gowin`, dok se neposredno pored njega (na istom nivou) nalazi zalijepljeni folder `sim`, namijenjen simulaciji.

Unutar foldera `sim` potrebno je izmijeniti dva fajla. Prvi je skripta `run-sim`, koju je najlakše otvoriti u nekom code ili tekst editoru (preporučuje se VS Code). U njoj je potrebno promijeniti vrijednost varijable `LAB_NAME` tako da odgovara imenu top modula – u ovom slučaju `zad1` – kao što je prikazano u Kodu 2. Ova varijabla se dalje koristi kroz cijelu skriptu za automatsko generisanje odgovarajućih Verilator naredbi, imena izlaznih fajlova i pokretanje simulacije, pa je njena tačna vrijednost preduslov da bi skripta uopšte mogla pronaći i kompajlirati odgovarajući modul.

Drugi fajl koji je potrebno prilagoditi jeste testbench fajl `versim.cpp`, prikazan u Kodu 3. Prva i obavezna izmjena jeste zamjena svih instanci `VLab1` sa `Vzad1` – ovo je posljedica

toga što Verilator prilikom kompajliranja generiše C++ klasu čiji naziv odgovara imenu top modula, sa prefiksom V (u ovom slučaju Vzad1), pa bez ove izmjene simulacija neće moći biti pokrenuta jer traženi tip klase ne bi postojao. Nakon toga je potrebno podesiti i vremenski tok simulacije tako da se jasno mogu razlučiti sve kombinacije ulaznih varijabli i pripadajući izlazi. Pošto se u ovom zadatku koriste samo tri ulazne varijable (A , B i C), svih osam mogućih kombinacija može se jednostavno definisati ručno, direktno unutar `versim.cpp` fajla, dodjeljivanjem odgovarajućih vrijednosti u unaprijed određenim vremenskim trenucima simulacije, kao što je vidljivo u kodu ispod.

Nakon pokretanja skripte `run-sim`, Verilator kompajlira SystemVerilog kod u C++ model, izvršava simulaciju u trajanju definisanom u `versim.cpp` fajlu, generiše `waveform.fst` fajl sa zapisom svih signala tokom simulacije, te automatski otvara GTKWave alat u kojem se dobijeni rezultati mogu vizuelno uporediti sa tablicom istine 1.

Kod 2 Skripta `run-sim` prilagođena za pokretanje simulacije zadatka 1.

```
LAB_NAME="zad1"

%-----

rm -rf output
mkdir output
cd output

verilator \
    --top-module ${LAB_NAME} \
    -Wall \
    +define+SIM_ONLY \
    --timescale 1ns/1ns \
    --trace-fst \
    --cc \
    ../../gowin/src/*.sv \
    --exe versim.cpp \
    --Mdir .

make -f V${LAB_NAME}.mk V${LAB_NAME}

./V${LAB_NAME}

%verilator ../../1.gowin/src/${LAB_NAME}.sv -xml-only --bbox-
    sys --Mdir .
%xml2stems V${LAB_NAME}.xml V${LAB_NAME}.stems
%gtkwave --stems V${LAB_NAME}.stems waveform.fst &

gtkwave waveform.fst
```

Kod 3 Testbench fajl `versim.cpp` sa definisanim testnim vektorima za sve kombinacije ulaza A , B i C .

```

#include <stdlib.h>
#include <iostream>
#include <verilated.h>
#include <verilated_fst_c.h>
#include "Vzad1.h"
#include "Vzad1___024root.h"

#define MAX_SIM_TIME 4000
vluint64_t sim_time = 0;

double sc_time_stamp() {
    return sim_time;
}

int main(int argc, char** argv, char** env) {
    Vzad1 *dut = new Vzad1;

    Verilated::traceEverOn(true);
    VerilatedFstC *m_trace = new VerilatedFstC;

    dut->trace(m_trace, 5);
    m_trace->open("waveform.fst");

    while (sim_time < MAX_SIM_TIME) {

        if(sim_time == 0)
        {
            dut->A = 0;
            dut->B = 0;
            dut->C = 0;
        }

        if(sim_time == 500)
        {
            dut->A = 1;
            dut->B = 0;
            dut->C = 0;
        }

        if(sim_time == 1000)
        {
            dut->A = 0;
            dut->B = 1;
            dut->C = 0;
        }

        if(sim_time == 1500)
    
```

```

        {
            dut->A = 1;
            dut->B = 1;
            dut->C = 0;
        }

        if(sim_time == 2000)
        {
            dut->A = 0;
            dut->B = 0;
            dut->C = 1;
        }

        if(sim_time == 2500)
        {
            dut->A = 1;
            dut->B = 0;
            dut->C = 1;
        }

        if(sim_time == 3000)
        {
            dut->A = 0;
            dut->B = 1;
            dut->C = 1;
        }

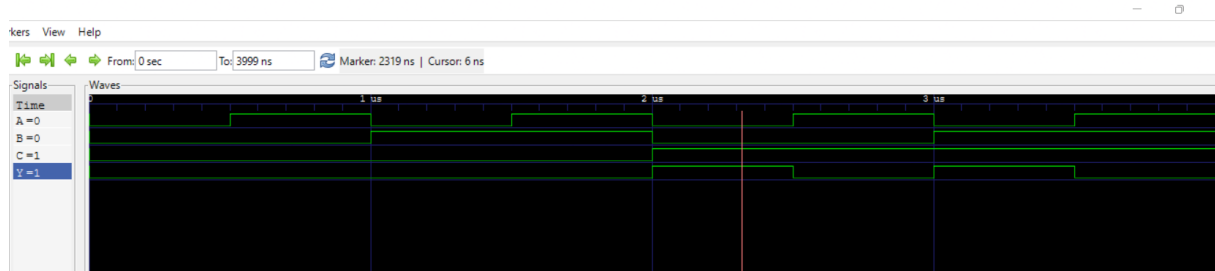
        if(sim_time == 3500)
        {
            dut->A = 1;
            dut->B = 1;
            dut->C = 1;
        }

        dut->eval();
        m_trace->dump(sim_time);
        sim_time++;
    }

    m_trace->close();
    delete dut;
    exit(EXIT_SUCCESS);
}

```

Na slici 1 prikazani su rezultati simulacije prvog zadatka. Može se uočiti da izlaz logičke funkcije poprima vrijednost logičke jedinice isključivo kada je $C = 1$ i $A = 0$, nezavisno od vrijednosti ulaza B . Prema tome, postoje dvije kombinacije ulaznih vrijednosti za koje je izlaz jednak logičkoj jedinici, što je u potpunosti u skladu s tablicom istine prikazanom u tabeli 1.



Slika 1: Simulacija za zadatak 1 iz laboratorijske vježbe broj 1.

Zadatak 2 – Implementacija kombinacione logičke funkcije sa pet promjenjivih

U okviru drugog zadatka potrebno je implementirati kombinacionu logičku funkciju sa pet ulaznih promjenjivih, definisanu izrazom:

$$Y = \bar{A}BCD \vee \overline{A \vee \bar{E}} \vee B\bar{C}D\bar{E} \quad (2)$$

gdje su A, B, C, D i E ulazne binarne varijable, a Y izlaz kola. Kao i u prethodnom zadatku, cilj je da student na osnovu datog izraza izvede tablicu istine, implementira funkciju u SystemVerilogu, te potvrdi ispravnost implementacije putem simulacije prije prebacivanja koda na FPGA razvojni sistem. Povećanjem broja ulaznih varijabli sa tri na pet, broj mogućih kombinacija ulaza raste sa $2^3 = 8$ na $2^5 = 32$, čime se student dodatno uvježbava u sistematičnom pristupu izvođenju tablice istine za funkcije sa većim brojem ulaza.

Primjenom De Morganovog zakona na drugi član izraza (2) dobija se $\overline{A \vee \bar{E}} = \bar{A}E$, čime se funkcija svodi na:

$$Y = \bar{A}BCD \vee \bar{A}E \vee B\bar{C}D\bar{E} \quad (3)$$

Iz pojednostavljenog izraza (3) se dodatno može uočiti da je izlaz Y jednak jedinici u sljedeća tri slučaja: kada je $A = 0$ i istovremeno $B = C = D = 1$; kada je $A = 0$ i $E = 1$ (nezavisno od preostalih varijabli); te kada je $B = 1, C = 0, D = 1$ i $E = 0$ (nezavisno od varijable A). Ovo zapažanje studentu služi kao brz i pouzdan način provjere ispravnosti tablice istine i kasnijih simulacionih rezultata, bez potrebe za provjerom svih 32 kombinacija ponaosob.

Tabela 2: Tablica istine za logičku funkciju $Y = \bar{A}BCD \vee A \vee \bar{E} \vee B\bar{C}D\bar{E}$.

A	B	C	D	E	Y
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	0
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	1

A	B	C	D	E	Y
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	0
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	1
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	0

Implementacija date logičke funkcije u SystemVerilogu, kao i u prethodnom zadatku, izvedena je korištenjem bloka `always_comb`, s obzirom na to da se i ovdje radi o čisto kombina-
cionoj logici bez memorijskog elementa:

Kod 4 SystemVerilog implementacija logičke funkcije zadatka 2.

```

1 module zad2 (
2     input logic A,
3     input logic B,
4     input logic C,
5     input logic D,
6     input logic E,
7     output logic Y
8 );
9
10 always_comb begin
11     Y = (~A & B & C & D) | ~(A | ~E) | (B & ~C & D & ~E);
12 end
13
14 endmodule

```

Nakon pisanja koda, potrebno je izvršiti simulaciju putem Verilator alata, uporediti dobijene rezultate sa tablicom istine 2, te tek nakon potvrde ispravnosti implementaciju prebaciti na Tang Nano 9K razvojni sistem radi konačnog testiranja na fizičkom hardveru.

Simulacija putem Verilator alata

Kao i u prethodnom zadatku, prije prebacivanja rješenja na fizički FPGA uređaj potrebno je verifikovati ispravnost napisanog koda putem simulacije. Postupak je identičan onom opisanom

u zadatku 1, s tim da je ovdje ključno da se `.sv` fajl kreiran tokom otvaranja projekta nazove `zad2`, u skladu sa imenom top modula. Nakon kreiranja projekta i pisanja koda koji opisuje logičku funkciju (2), folder `sim` se ponovo kopira iz test fajla u direktorij projekta, na isti način kao u zadatku 1.

Unutar foldera `sim` ponovo je potrebno izmijeniti dva fajla. U skripti `run-sim` varijabla `LAB_NAME` se mijenja u `zad2`, kao što je prikazano u Kodu 5. U testbench fajlu `versim.cpp`, prikazanom u Kodu 6, sve instance `Vzad1` (ili `VLab1`, u zavisnosti od toga sa koje se osnove kreće) potrebno je zamijeniti sa `Vzad2`, iz istog razloga kao i ranije – naziv generisane Verilator C++ klase mora odgovarati imenu top modula.

Dodatna razlika u odnosu na zadatak 1 proizilazi iz činjenice da funkcija sada ima pet ulaznih varijabli (*A*, *B*, *C*, *D* i *E*) umjesto tri, čime broj mogućih kombinacija raste sa 8 na 32. U skladu s tim, testbench fajl je proširen tako da definiše svih 32 kombinacija ulaznih vrijednosti, svaku u trajanju od 500 ns, čime je ukupno vrijeme trajanja simulacije (`MAX_SIM_TIME`) produženo na 16000 ns kako bi sve kombinacije stale u simulaciju. Redoslijed kombinacija u testbenchu odgovara redoslijedu kombinacija u tablici istine 2 (standardno binarno brojanje od 00000 do 11111), čime se rezultati simulacije mogu direktno, red po red, uporediti sa tablicom istine.

Kod 5 Skripta `run-sim` prilagođena za pokretanje simulacije zadatka 2.

```
LAB_NAME="zad2"

%-----

rm -rf output
mkdir output
cd output

verilator \
  --top-module ${LAB_NAME} \
  -Wall \
  +define+SIM_ONLY \
  --timescale 1ns/1ns \
  --trace-fst \
  --cc \
  ../../gowin/src/*.sv \
  --exe versim.cpp \
  --Mdir .

make -f V${LAB_NAME}.mk V${LAB_NAME}

./V${LAB_NAME}

gtkwave waveform.fst
```

Kod 6 Testbench fajl `versim.cpp` sa definisanim testnim vektorima za svih 32 kombinacije ulaza *A*, *B*, *C*, *D* i *E*.

```

#include <stdlib.h>
#include <iostream>
#include <verilated.h>
#include <verilated_fst_c.h>
#include "Vzad2.h"
#include "Vzad2___024root.h"

#define MAX_SIM_TIME 16000
vluint64_t sim_time = 0;

double sc_time_stamp() {
    return sim_time;
}

int main(int argc, char** argv, char** env) {
    Vzad2 *dut = new Vzad2;

    Verilated::traceEverOn(true);
    VerilatedFstC *m_trace = new VerilatedFstC;

    dut->trace(m_trace, 5);
    m_trace->open("waveform.fst");

    while (sim_time < MAX_SIM_TIME) {

        if(sim_time == 0)
        {
            dut->A = 0;
            dut->B = 0;
            dut->C = 0;
            dut->D = 0;
            dut->E = 0;
        }

        if(sim_time == 500)
        {
            dut->A = 0;
            dut->B = 0;
            dut->C = 0;
            dut->D = 0;
            dut->E = 1;
        }

        if(sim_time == 1000)
        {
            dut->A = 0;
            dut->B = 0;
            dut->C = 0;
        }
    }
}

```

```

        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 1500)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 0;
        dut->D = 1;
        dut->E = 1;
    }

    if(sim_time == 2000)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 1;
        dut->D = 0;
        dut->E = 0;
    }

    if(sim_time == 2500)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 1;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 3000)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 1;
        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 3500)
    {
        dut->A = 0;
        dut->B = 0;
        dut->C = 1;
        dut->D = 1;
        dut->E = 1;
    }

```

```

if(sim_time == 4000)
{
    dut->A = 0;
    dut->B = 1;
    dut->C = 0;
    dut->D = 0;
    dut->E = 0;
}

if(sim_time == 4500)
{
    dut->A = 0;
    dut->B = 1;
    dut->C = 0;
    dut->D = 0;
    dut->E = 1;
}

if(sim_time == 5000)
{
    dut->A = 0;
    dut->B = 1;
    dut->C = 0;
    dut->D = 1;
    dut->E = 0;
}

if(sim_time == 5500)
{
    dut->A = 0;
    dut->B = 1;
    dut->C = 0;
    dut->D = 1;
    dut->E = 1;
}

if(sim_time == 6000)
{
    dut->A = 0;
    dut->B = 1;
    dut->C = 1;
    dut->D = 0;
    dut->E = 0;
}

if(sim_time == 6500)
{

```

```

        dut->A = 0;
        dut->B = 1;
        dut->C = 1;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 7000)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 1;
        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 7500)
    {
        dut->A = 0;
        dut->B = 1;
        dut->C = 1;
        dut->D = 1;
        dut->E = 1;
    }

    if(sim_time == 8000)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 0;
        dut->D = 0;
        dut->E = 0;
    }

    if(sim_time == 8500)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 0;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 9000)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 0;
    }

```

```

        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 9500)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 0;
        dut->D = 1;
        dut->E = 1;
    }

    if(sim_time == 10000)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 1;
        dut->D = 0;
        dut->E = 0;
    }

    if(sim_time == 10500)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 1;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 11000)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 1;
        dut->D = 1;
        dut->E = 0;
    }

    if(sim_time == 11500)
    {
        dut->A = 1;
        dut->B = 0;
        dut->C = 1;
        dut->D = 1;
        dut->E = 1;
    }

```

```

if(sim_time == 12000)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 0;
    dut->D = 0;
    dut->E = 0;
}

if(sim_time == 12500)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 0;
    dut->D = 0;
    dut->E = 1;
}

if(sim_time == 13000)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 0;
    dut->D = 1;
    dut->E = 0;
}

if(sim_time == 13500)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 0;
    dut->D = 1;
    dut->E = 1;
}

if(sim_time == 14000)
{
    dut->A = 1;
    dut->B = 1;
    dut->C = 1;
    dut->D = 0;
    dut->E = 0;
}

if(sim_time == 14500)
{

```

```

        dut->A = 1;
        dut->B = 1;
        dut->C = 1;
        dut->D = 0;
        dut->E = 1;
    }

    if(sim_time == 15000)
    {
        dut->A = 1;
        dut->B = 1;
        dut->C = 1;
        dut->D = 1;
        dut->E = 0;
    }

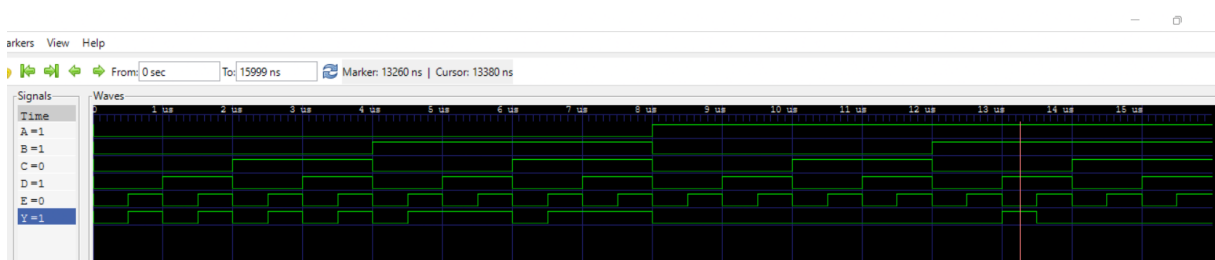
    if(sim_time == 15500)
    {
        dut->A = 1;
        dut->B = 1;
        dut->C = 1;
        dut->D = 1;
        dut->E = 1;
    }

    dut->eval();
    m_trace->dump(sim_time);
    sim_time++;
}

m_trace->close();
delete dut;
exit(EXIT_SUCCESS);
}

```

Na slici 2 prikazani su rezultati simulacije drugog zadatka. Izlaz Y poprima vrijednost logičke jedinice tačno u onim vremenskim intervalima koji odgovaraju kombinacijama ulaza za koje je, prema pojednostavljenom izrazu (3), ispunjen bar jedan od uslova $\bar{A}BCD$, $\bar{A}E$ ili $B\bar{C}\bar{D}\bar{E}$, što je u potpunosti u skladu s tablicom istine prikazanom u tabeli 2.



Slika 2: Simulacija za zadatak 2 iz laboratorijske vježbe jedan